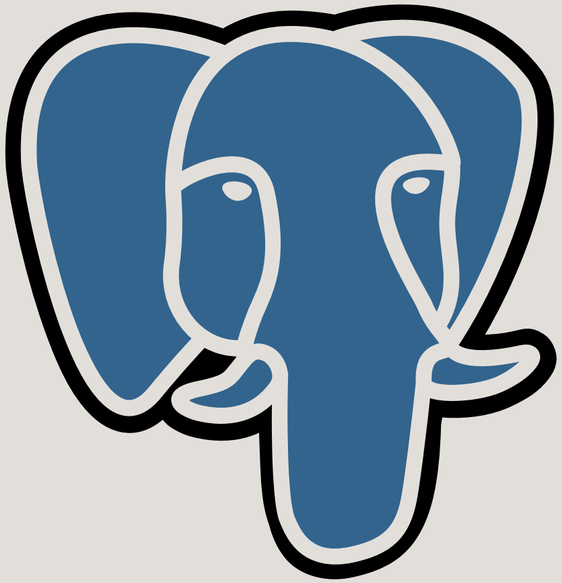
Gestão de serviços para oficina mecânica

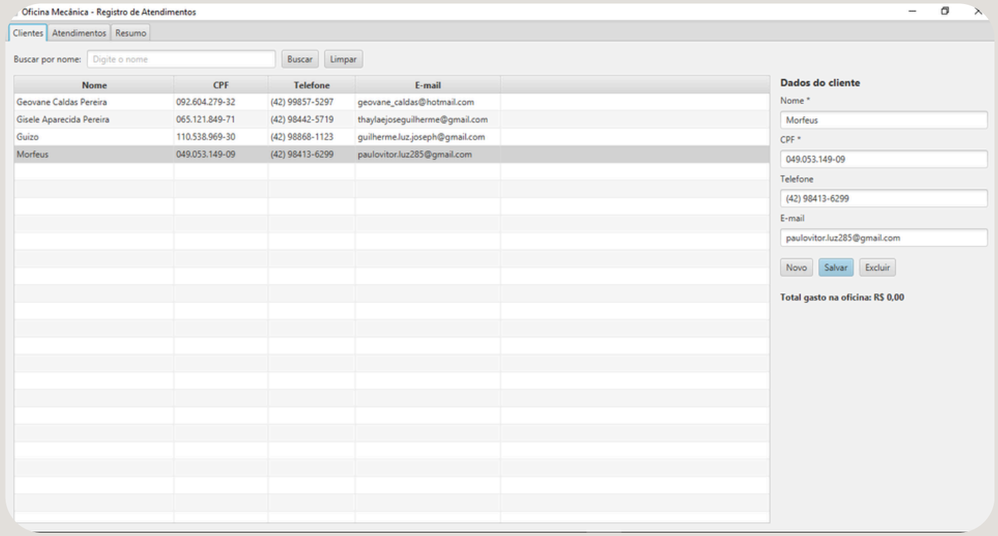




José Guilherme Pereira Da Luz | Universidade Estadual Do Centro-Oeste | Programação Orientada a objetos II

Sistema de gestão de serviços com foco em oficinas mecânicas

- Cadastro de clientes
- Ordens de serviço
- Ticket médio e resumo financeiro





Padrão arquitetural MVC

View

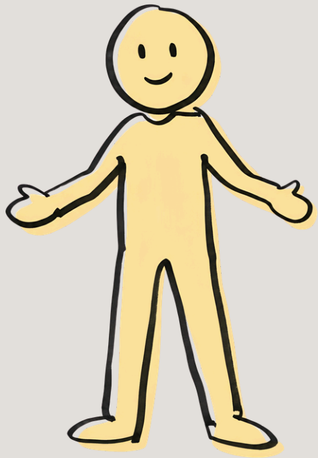
Telas JavaFX em arquivos .fxml: Clientes, Atendimentos e Resumo.

Controller

Recebe as ações do usuário e chama a camada de serviço.

Model

Entidades Cliente, Atendimento e Servico, mais o acesso ao banco.



Usuário

View atualizada

MVC com Service Layer e DAO

View (FXML)	Telas de clientes, atendimentos e resumo
Controller	Recebe os cliques e chama a Service
Service	Valida os dados e aplica as regras de negócio
DAO (model.dao)	Interfaces e classes JDBC com PreparedStatement
PostgreSQL	**Tabelas cliente, atendimento e serviço**

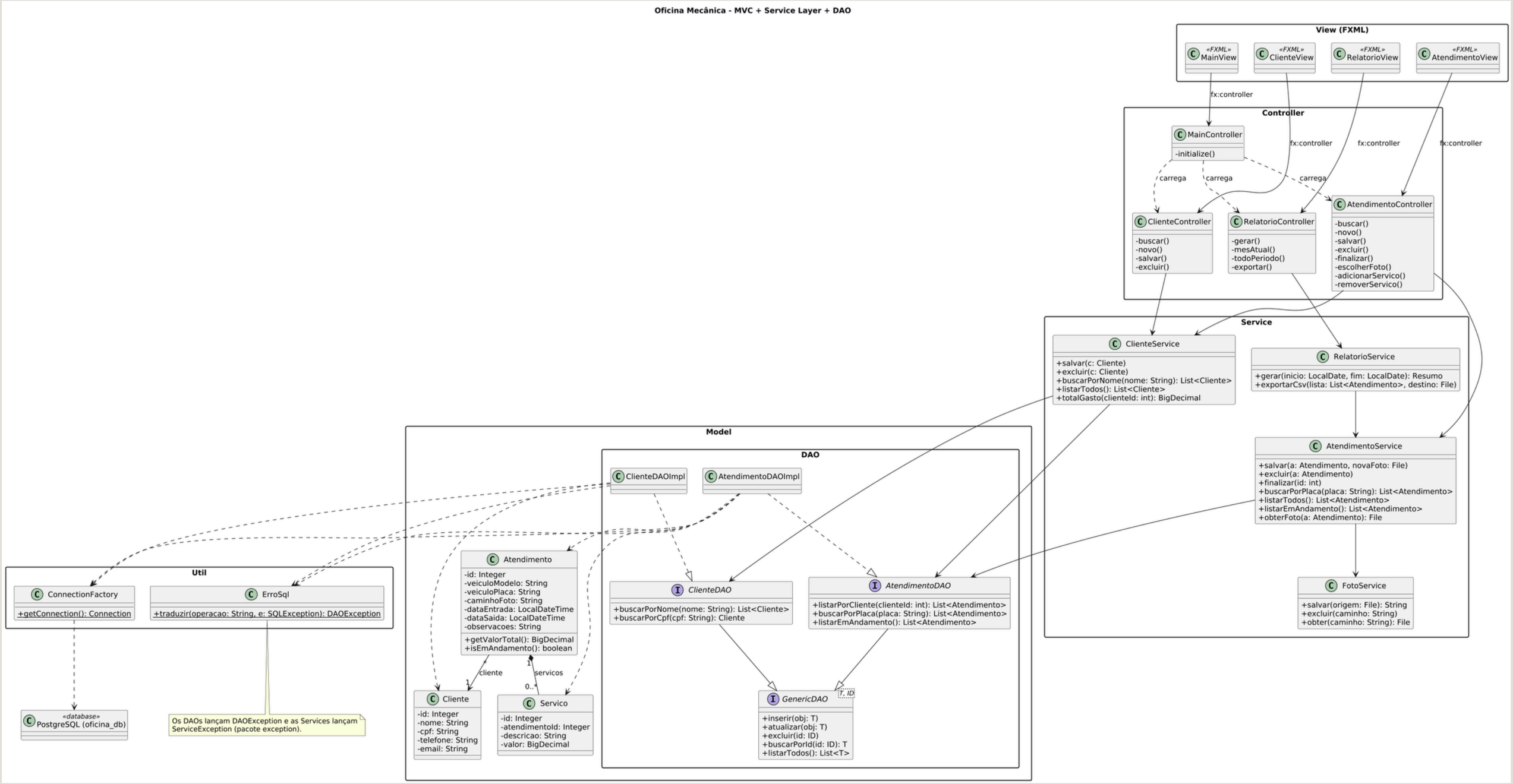
Funcionalidades do sistema

Clientes

- Cadastrar novos clientes
- Consultar e buscar por nome
- Editar e excluir registros
- Ver o total gasto por cliente

Atendimentos

- Veículo, placa e foto
- Data e hora de entrada e saída
- Trabalhos realizados com valores
- Total gasto calculado automaticamente


Oficina Mecânica - MVC + Service Layer + DAO
View (FXML)
«FXML» MainView
«FXML» ClienteView
«FXML» RelatorioView
«FXML» AtendimentoView
fx:controller
Controller
MainController
-initialize()
carrega
ClienteController
-buscar()
-novo()
-salvar()
-excluir()
RelatorioController
-gerar()
-mesAtual()
-todoPeriodo()
-exportar()
AtendimentoController
-buscar()
-novo()
-salvar()
-excluir()
-finalizar()
-escolherFoto()
-adicionarServico()
-removerServico()
Service
ClienteService
+salvar(c: Cliente)
+excluir(c: Cliente)
+buscarPorNome(nome: String): List<Cliente>
+listarTodos(): List<Cliente>
+totalGasto(clienteId: int): BigDecimal
RelatorioService
+gerar(inicio: LocalDate, fim: LocalDate): Resumo
+exportarCsv(lista: List<Atendimento>, destino: File)
AtendimentoService
+salvar(a: Atendimento, novaFoto: File)
+excluir(a: Atendimento)
+finalizar(id: int)
+buscarPorPlaca(placa: String): List<Atendimento>
+listarTodos(): List<Atendimento>
+listarEmAndamento(): List<Atendimento>
+obterFoto(a: Atendimento): File
FotoService
+salvar(origem: File): String
+excluir(caminho: String)
+obter(caminho: String): File
Model
DAO
ClienteDAOImpl
AtendimentoDAOImpl
ClienteDAO
+buscarPorNome(nome: String): List<Cliente>
+buscarPorCpf(cpf: String): Cliente
AtendimentoDAO
+listarPorCliente(clienteId: int): List<Atendimento>
+buscarPorPlaca(placa: String): List<Atendimento>
+listarEmAndamento(): List<Atendimento>
GenericDAO
T, ID
+inserir(obj: T)
+atualizar(obj: T)
+excluir(id: ID)
+buscarPorId(id: ID): T
+listarTodos(): List<T>
Atendimento
-id: Integer
-veiculoModelo: String
-veiculoPlaca: String
-caminhoFoto: String
-dataEntrada: LocalDateTime
-dataSaida: LocalDateTime
-observacoes: String
+getValorTotal(): BigDecimal
+isEmAndamento(): boolean
cliente
servicos
Cliente
-id: Integer
-nome: String
-cpf: String
-telefone: String
-email: String
Servico
-id: Integer
-atendimentoId: Integer
-descricao: String
-valor: BigDecimal
Util
ConnectionFactory
+getConnection(): Connection
ErroSql
+traduzir(operacao: String, e: SQLException): DAOException
«database» PostgreSQL (oficina_db)
Os DAOs lançam DAOException e as Services lançam ServiceException (pacote exception).